

Aula 02 — Spring Framework

HTML/JS + API JSON ù Projeto StockSales

Mauricio Duailibi Neto

Turma Java Entra21

Como será a aula de hoje

Duas partes, com intervalo no meio:

Parte 1 — 18h30 às 20h

- Revisão da Aula 01
- Estrutura de um projeto Spring Boot
- Como o HTML conversa com o backend
- Exercícios guiados

Parte 2 — 20h20 às 22h

- Apresentação do projeto do curso: **StockSales**
- Configuração inicial + primeira página + listar produtos

Mapa da aula

- 1 Revisão da Aula 01
- 2 Arquitetura de um projeto Spring Boot
- 3 HTML estático + API JSON
- 4 Exercícios — Parte 1
- 5 Parte 2 — O projeto StockSales
- 6 Mão na massa — StockSales
- 7 Recapitulando

O que construímos na Aula 01

Na última aula, o foco foi fazer o Spring responder no navegador:

- 1 Instalar JDK 21 + VS Code + extensões
- 2 Gerar um projeto no `start.spring.io`
- 3 Criar um `@RestController`
- 4 Responder texto em endereços como `/` e `/ola`
- 5 Usar `@RequestParam`, por exemplo `/saudacao?nome=Ana`

Lembrete

O navegador pede uma URL → o Controller responde.

Anotações que precisam ficar na cabeça

`@RestController` Diz: “esta classe recebe pedidos HTTP”.

`@GetMapping("/caminho")` Diz: “quando alguém acessar este caminho com GET, rode este método”.

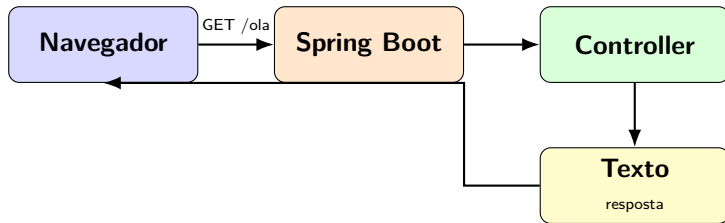
`@RequestParam` Lê valores da URL depois do ?, por exemplo `?nome=Ana`.

`pom.xml` Lista de dependências do Maven, por exemplo Spring Web.

Sem isso, o restante da disciplina fica mais difícil.

Com isso, vocês já conseguem conversar com o servidor.

Fluxo que já vimos



Isso funciona — mas a resposta era **texto puro**.
Hoje vamos dar o próximo passo: **páginas de verdade**.

O limite do texto puro

Na Aula 01, o navegador mostrava algo assim:

```
Olá, Spring Boot! Bem-vindo à disciplina.
```

Isso é ótimo para aprender o fluxo.

Mas um sistema real precisa de:

- Layout: cores, títulos, botões
- Listas, tabelas, formulários
- Interação: clicar e atualizar a tela

Para isso, usamos **HTML + CSS + JavaScript**.

Como o navegador mostra uma **página**
e, ao mesmo tempo, busca **dados**
que vêm do Spring?

A resposta curta:

HTML/JS estático + API JSON.

Vamos entender isso com calma.

Antes de HTML: onde cada coisa mora

Um projeto Spring Boot não é “um arquivo só”.
É uma **casinha organizada**.

Se vocês souberem **onde procurar**,
programar fica muito menos assustador.

Meta deste bloco

Olhar a estrutura de pastas e saber dizer:
“isso é código Java” ã “isso é página” ã “isso é dependência”.

Estrutura típica do projeto

```
meu-projeto/  
  pom.xml  
  src/  
    main/  
      java/  
        com/entra21/...  
          App.java  
          ...Controller.java  
      resources/  
        application.properties  
        static/          HTML, CSS, JS  
target/                  gerado ao compilar
```

Hoje a pasta nova mais importante é: `static/`.

O que cada pasta faz

`pom.xml` “Lista de compras” do projeto: quais bibliotecas baixar.

`src/main/java` Código Java: Controllers, regras, etc.

`src/main/resources` Configuração e arquivos da aplicação.

`static/` Páginas e estilos que o navegador baixa: `.html`, `.css`, `.js`, imagens.

`target/` Resultado da compilação — **não editamos** na mão.

Duas portas no mesmo servidor

No Spring Boot, o mesmo programa — porta 8080 — pode servir:

Páginas

```
/
/index.html
/css/estilo.css
/js/app.js
```

Arquivos da pasta static/.

API — dados

```
/api/produtos
/api/clientes
/api/vendas
```

Métodos do @RestController.

Frase de ouro

A página é a **vitrine**.

A API é o **estoque de dados** por trás da vitrine.

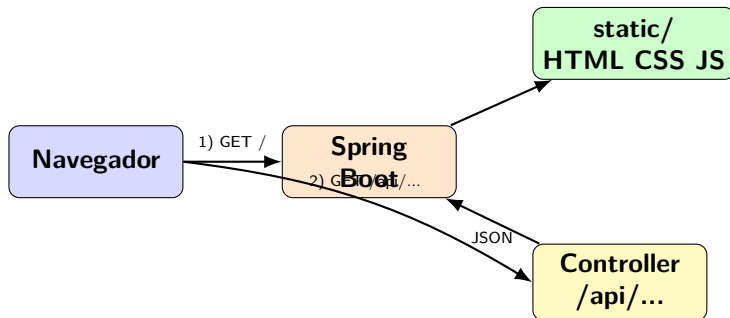
Analogia do restaurante

- **HTML** = o cardápio e a mesa: o que o cliente vê
- **CSS** = a decoração: cores, fontes, espaçamento
- **JavaScript** = o garçom: vai à cozinha e traz o pedido
- **API Spring** = a cozinha: prepara e devolve os dados
- **JSON** = a bandeja com o prato organizado

O usuário quase nunca “entra na cozinha”.

Ele conversa com a interface — e a interface fala com a API.

Diagrama: página e API juntas



- 1 O navegador baixa a página `index.html`.
- 2 O JavaScript da página chama a API e recebe dados.

Por que separar página e API

Porque isso facilita a vida no dia a dia:

- A tela cuida de mostrar as informações
- O backend cuida de devolver os dados
- Cada parte fica mais fácil de entender e testar
- O mesmo backend pode alimentar várias telas depois

No nosso curso

Vamos guardar o HTML/CSS/JS na pasta `static/`
e os dados nos Controllers em caminhos `/api/....`

Classes Java que vão aparecer bastante

Application Classe com `main` — sobe o Spring.

Controller Recebe o pedido HTTP e devolve a resposta.

Classe de dados Representa algo do sistema, por exemplo Produto ou Aluno.

Hoje: **Application + Controller + classe de dados + HTML/JS.**

Sem banco de dados ainda.

Ainda **não**.

Nas primeiras aulas do StockSales, os dados ficam em uma **lista na memória** do Java.

- Reiniciou o servidor? Os dados voltam ao que estava no código.
- Isso é proposital: aprendemos a API e a tela primeiro.
- Banco entra mais à frente no curso.

O que é HTML estático no Spring

“Estático” aqui significa:

- O arquivo já existe em `src/main/resources/static/`
- O Spring **entrega o arquivo** para o navegador
- Não precisamos criar um Controller só para servir o HTML

Exemplos:

Arquivo	URL
<code>static/index.html</code>	<code>http://localhost:8080/</code>
<code>static/produtos.html</code>	<code>.../produtos.html</code>
<code>static/css/estilo.css</code>	<code>.../css/estilo.css</code>
<code>static/js/app.js</code>	<code>.../js/app.js</code>

Exemplo mínimo de index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>StockSales</title>
  <link rel="stylesheet" href="/css/estilo.css">
</head>
<body>
  <h1>StockSales</h1>
  <p>Bem-vindo ao sistema da turma!</p>
  <ul id="lista-produtos"></ul>

  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
  <script src="/js/app.js"></script>
</body>
</html>
```

O que é JSON

JSON = jeito padrão de **organizar dados em texto** para programas trocarem informação.

Parece um objeto Java, mas é texto:

```
{"nome": "Teclado", "preco": 150.0, "estoque": 10}
```

- Chaves entre aspas: "nome"
- Dois-pontos separa chave e valor
- Texto também entre aspas; números sem aspas
- Lista: [..., ...]

JSON de uma lista de produtos

```
[
  {
    "id": 1,
    "nome": "Teclado Mecanico",
    "preco": 250.0,
    "estoque": 12
  },
  {
    "id": 2,
    "nome": "Mouse Gamer",
    "preco": 120.0,
    "estoque": 30
  }
]
```

O JavaScript sabe ler isso.

O Spring gera isso a partir dos objetos Java.

No Java: a classe Produto

Criamos uma classe normal, com atributos e get/set:

```
public class Produto {  
    private Long id;  
    private String nome;  
    private double preco;  
    private int estoque;  
  
    public Produto(Long id, String nome,  
                   double preco, int estoque) {  
        this.id = id;  
        this.nome = nome;  
        this.preco = preco;  
        this.estoque = estoque;  
    }  
  
    public Long getId() { return id; }  
    public String getNome() { return nome; }  
    public double getPreco() { return preco; }  
    public int getEstoque() { return estoque; }
```

Por que os métodos get

O Spring monta o JSON usando os get....

Método Java	Campo no JSON
getNome()	"nome"
getPreco()	"preco"
getEstoque()	"estoque"

Sem o get, o campo quase não aparece no JSON.

Por isso: atributos `private` + get públicos.

Controller que devolve JSON

```
@RestController
public class ProdutoController {

    @GetMapping("/api/produtos")
    public List<Produto> listar() {
        List<Produto> produtos = new ArrayList<>();
        produtos.add(new Produto(1L, "Teclado", 250.0, 12));
        produtos.add(new Produto(2L, "Mouse", 120.0, 30));
        return produtos;
    }
}
```

Importes que costumam faltar:

```
import java.util.ArrayList;
import java.util.List;
```

Teste: <http://localhost:8080/api/produtos>

Por que o caminho começa com /api

É uma **convenção** nossa — um combinado claro:

- /api/... → dados em JSON
- /, /produtos.html → páginas

Assim ninguém confunde:

- “Estou pedindo a página?”
- “Ou estou pedindo os dados?”

Como o JS fala com a API

Precisamos de uma ferramenta no JavaScript que:

- 1 Chame um endereço `/api/...`
- 2 Receba o JSON
- 3 Entregue esses dados para a gente usar na tela

No curso vamos usar o **Axios**.

Em português

Axios = biblioteca pronta para fazer pedidos HTTP de um jeito simples e organizado.

Como incluir o Axios na página

Não vamos instalar nada no computador.

Colocamos o Axios pela internet, com uma tag script:

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>  
<script src="/js/app.js"></script>
```

Ordem importa

- 1) Axios primeiro
- 2) Nosso app.js depois

Se inverter, o app.js tenta usar o Axios antes dele existir — e quebra.

Primeiro pedido com Axios

Buscar a lista de produtos:

```
axios.get("/api/produtos")  
  .then(function (resposta) {  
    console.log(resposta.data);  
  });
```

`axios.get` Pede dados com GET.

`resposta.data` Já vem o JSON convertido.

Não precisamos chamar `.json()` na mão.

Leitura em voz alta

“Axios, busca /api/produtos.

Quando voltar, me mostra resposta.data.”

Montando a lista na tela

```
var ul = document.getElementById("lista-produtos");

axios.get("/api/produtos")
  .then(function (resposta) {
    var produtos = resposta.data;
    ul.innerHTML = "";
    for (var i = 0; i < produtos.length; i++) {
      var p = produtos[i];
      var li = document.createElement("li");
      li.textContent = p.nome + " - R$ " + p.preco
        + " - estoque: " + p.estoque;
      ul.appendChild(li);
    }
  });
```

Lembra do @RequestParam?

Na Aula 01, o backend já lia parâmetros na URL:

```
/saudacao?nome=Ana
```

O ? começa a lista de parâmetros.

O & separa vários parâmetros:

```
/soma?a=2&b=3
```

Agora vamos aprender a **enviar** esses parâmetros a partir do JavaScript, com Axios.

O que é “enviar parâmetros”

Às vezes a API precisa de informação extra, por exemplo:

- Qual o **nome** da pessoa?
- Qual o **id** do produto?
- Qual o texto da **busca**?

Esses valores vão na URL, depois do ?.

Exemplo

```
/api/saudacao?nome=Ana
```

O parâmetro se chama nome.

O valor é Ana.

Backend: receber o parâmetro

No Controller, igual à Aula 01:

```
@GetMapping("/api/saudacao")
public Mensagem saudacao(
    @RequestParam String nome) {
    return new Mensagem("Ola, " + nome + "!");
}
```

Teste manual no navegador:

<http://localhost:8080/api/saudacao?nome=Ana>

JSON esperado:

```
{"texto": "Ola, Ana!"}
```


Jeito 1: colocar na URL na mão

Vocês podem montar a URL juntando texto:

```
var nome = "Ana";

axios.get("/api/saudacao?nome=" + nome)
  .then(function (resposta) {
    console.log(resposta.data.texto);
  });
```

Funciona.

Mas fica confuso quando tem **vários** parâmetros.

Jeito 2: objeto params do Axios

O Axios tem um jeito mais organizado:

```
axios.get("/api/saudacao", {  
  params: {  
    nome: "Ana"  
  }  
})  
  .then(function (resposta) {  
    console.log(resposta.data.texto);  
  });
```

O Axios transforma isso em:

/api/saudacao?nome=Ana

Use este jeito

Menos erro de digitação.

Mais fácil de ler e de acrescentar parâmetros.

Vários parâmetros de uma vez

```
axios.get("/api/soma", {  
  params: {  
    a: 2,  
    b: 3  
  }  
})  
  .then(function (resposta) {  
    console.log(resposta.data);  
  });
```

Vira na prática:

`/api/soma?a=2&b=3`

No Java:

```
@RequestParam int a, @RequestParam int b
```

Parâmetro vindo de um input

Na página:

```
<input id="campo-nome" type="text">  
<button id="btn-saudacao">Saudar</button>  
<p id="saida"></p>
```

No app.js:

```
var botao = document.getElementById("btn-saudacao");  
var campo = document.getElementById("campo-nome");  
var saida = document.getElementById("saida");  
  
botao.addEventListener("click", function () {  
    axios.get("/api/saudacao", {  
        params: { nome: campo.value }  
    }).then(function (resposta) {  
        saida.textContent = resposta.data.texto;  
    });  
});
```

Lendo o fluxo com calma

- 1 A pessoa digita Ana no input
- 2 Clica no botão
- 3 Axios chama `/api/saudacao?nome=Ana`
- 4 O Spring lê nome com `@RequestParam`
- 5 Devolve JSON: `{"texto": "Ola, Ana!"}`
- 6 O JS pega `resposta.data.texto` e mostra na tela

Frase para lembrar

params no Axios ↔ `@RequestParam` no Spring.

GET com parâmetros × só GET

Situação	Exemplo
Listar tudo	GET /api/produtos
Buscar com filtro	GET /api/produtos?nome=Mouse
Saudação personalizada	GET /api/saudacao?nome=Ana

Parâmetro na URL = informação **curta** para filtrar ou personalizar a consulta.

Mais à frente veremos o POST, usado para **enviar dados maiores** e criar registros.

Fluxo completo

- 1 Usuário abre `http://localhost:8080/`
- 2 Spring entrega `index.html`, CSS, Axios e JS
- 3 O JS executa `axios.get("/api/produtos")`
- 4 Spring roda o método do Controller
- 5 Volta JSON com a lista
- 6 JS cria elementos `<li>` e mostra na página

Aula 02 em uma frase

Página na pasta `static/` + dados via `/api/...`

Ferramenta de detetive: aba Network

No navegador: **F12** ou botão direito → Inspecionar.

Aba **Network** / Rede.

Vocês conseguem ver:

- Se `index.html` carregou
- Se `/api/produtos` foi chamado
- Se os parâmetros aparecem na URL
- Qual JSON voltou
- Se deu erro 404 ou 500

Dica

Antes de pedir ajuda, olhem a aba Network.

Muitas respostas estão ali.

- **axios is not defined** — esqueceu o script do Axios, ou colocou depois do `app.js`
- **Página em branco** — veja o Console no F12
- **404** — caminho do `@GetMapping` diferente do `axios.get`
- **Parâmetro null no Java** — nome em `params` diferente do `@RequestParam`
- **JSON sem os campos** — faltou o `get` na classe
- **Mudança não apareceu** — reinicie o Spring

Hora de praticar

Vamos montar um **Painel da Turma** em 4 passos.

Cada exercício **completa** o anterior.

Use um projetinho novo no `start.spring.io` ou o `ola-spring`.

O que o painel terá no final

- Uma mensagem vinda da API
- Uma lista de alunos vinda da API

Salve → reinicie o Spring → teste.

Use o navegador e o F12.

Visão geral dos 4 passos

- 1 Criar a **página** com os espaços prontos e o script do Axios
- 2 Criar a classe Mensagem e o endpoint `/api/mensagem`
- 3 Ligar o **botão** ao `axios.get` da mensagem
- 4 Criar a classe Aluno, o endpoint `/api/alunos` e mostrar a lista

No final, a mesma página usa **dois** endpoints.
É o mesmo raciocínio do StockSales.

Exercício 1 — A página do painel

Criem:

`src/main/resources/static/index.html`

`src/main/resources/static/js/app.js`

o `app.js` pode começar vazio

Na página, já deixem estes elementos:

```
<h1>Painel da Turma</h1>
<button id="btn-mensagem">Buscar mensagem</button>
<p id="saida">Clique no botao...</p>
<h2>Alunos</h2>
<ul id="lista-alunos"></ul>
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
<script src="/js/app.js"></script>
```

Teste: `http://localhost:8080/`

A página abre — ainda sem dados da API.

Exercício 2 — Classe Mensagem e API

Criem a classe Mensagem com o atributo texto, construtor e getTexto().

No Controller:

```
@GetMapping("/api/mensagem")  
public Mensagem mensagem() {  
    return new Mensagem("Ola da API Spring!");  
}
```

Teste **ainda no navegador**, sem JS:

<http://localhost:8080/api/mensagem>

Esperado algo como:

```
{"texto": "Ola da API Spring!"}
```

Exercício 3 — Botão busca a mensagem

Continuando a **mesma** página do Exercício 1.

No `app.js`:

```
var botao = document.getElementById("btn-mensagem");
var saida = document.getElementById("saida");

botao.addEventListener("click", function () {
  axios.get("/api/mensagem")
    .then(function (resposta) {
      saida.textContent = resposta.data.texto;
    });
});
```

Clicou → a frase da API aparece no parágrafo.

Exercício 4 — Lista de alunos

Ainda no **mesmo** projeto e na **mesma** página.

- 1) Classe Aluno com nome e turma + construtor + gets
- 2) Endpoint:

```
@GetMapping("/api/alunos")
public List<Aluno> alunos() {
    List<Aluno> lista = new ArrayList<>();
    lista.add(new Aluno("Ana", "Entra21"));
    lista.add(new Aluno("Bruno", "Entra21"));
    lista.add(new Aluno("Carla", "Entra21"));
    return lista;
}
```

- 3) No app.js, ao carregar a página: `axios.get("/api/alunos")` e preencher `#lista-alunos` com `resposta.data`.

Painel pronto — o que vocês montaram

Uma única tela, dois pedidos à API:

- 1 Clique no botão → `/api/mensagem`
- 2 Ao abrir a página → `/api/alunos`

Isso é o ensaio do StockSales

Depois, no projeto do curso, a ideia é a mesma:
página HTML + Axios + lista vinda do Spring.

Checklist se travar

A pasta é `src/main/resources/static/`?

O script do Axios vem **antes** do `app.js`?

O endpoint começa com `/api/`?

O Controller tem `@RestController`?

A classe de dados tem os métodos `get`?

Os dados estão em `resposta.data`?

Reinicie o Spring depois de salvar?

Olhei o Console e a aba Network no F12?

Dúvida? Chamem o professor — mostrem a tela do F12.

Até já — retomamos às **20h20**

Segunda parte: o projeto **StockSales**
O sistema que vamos evoluir até o fim do curso

Bem-vindos à Parte 2

A partir de agora, quase toda aula terá:

- 1 Um pedaço teórico / exercício curto
- 2 Evolução do **mesmo projeto**

Objetivo: sentir como é trabalhar em um sistema de verdade, desde o começo até login, vendas e banco.

Nome do projeto

StockSales — vendas com controle de estoque.

Que problema o StockSales resolve

Imaginem uma loja simples:

- Tem **produtos** com preço e quantidade em estoque
- Tem **clientes** cadastrados
- Registra **vendas**
- Quando vende, o **estoque precisa baixar**

Esse tipo de sistema aparece bastante em empresas:
loja, estoque, atendimento, sistemas internos.

As três ideias centrais

Produto

Nome
Preço
Estoque

Cliente

Nome
E-mail
Telefone

Venda

Cliente
Itens
Total
Data

A parte mais interessante:
vender → **estoque diminui**.

Regra de ouro do estoque

Ao confirmar uma venda:

- 1 Verificar se todo item tem estoque suficiente
- 2 Se **faltar** estoque em qualquer item:
recusar a venda inteira
- 3 Se estiver ok: gravar a venda e diminuir o estoque

Importante

Não vendemos “metade”.

Ou a venda fecha toda, ou não fecha.

Essa regra será implementada em aulas futuras.

Telas que o sistema terá

- 1 **Home** — resumo do sistema
- 2 **Produtos** — listar e depois cadastrar/editar
- 3 **Clientes** — cadastro de clientes
- 4 **Nova venda** — escolher cliente + produtos
- 5 **Histórico de vendas**
- 6 **Login**
- 7 **Relatórios** — totais, estoque baixo

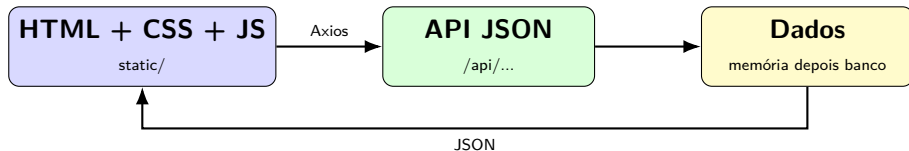
Hoje

Só a base: projeto no ar + home + **listar produtos**.

Mapa das próximas aulas

Aula	Foco no StockSales
2	Projeto + página + listar produtos
3	CRUD produtos e clientes
4	Vendas + baixa de estoque
5	Organização do código e validação
6	Banco de dados
7	Autenticação / login
8	Perfis: admin e vendedor
9	Relatórios, busca, estoque baixo
10	Fechamento + desafio final

Arquitetura que vamos usar



Página na static/. Dados no Controller. JSON no meio.

API inicial de hoje

Hoje precisamos de **um** endpoint:

GET /api/produtos

Resposta: lista JSON de produtos
com id, nome, preço e estoque.

Nas próximas aulas entram criação, edição, exclusão, clientes, vendas e login.

O que não vamos fazer hoje

Para não misturar demais:

- Cadastro e edição de produtos — Aula 3
- Clientes e vendas
- Banco de dados
- Login / senha
- Layout muito elaborado

Meta da noite

StockSales sobe.

Home abre.

Produtos aparecem vindo da API.

Passo 1 — Criar o projeto

Em `https://start.spring.io`:

- **Project:** Maven
- **Language:** Java
- **Spring Boot:** versão padrão do site
- **Group:** `com.entra21`
- **Artifact:** `stocksales`
- **Package:** `com.entra21.stocksales`
- **Packaging:** Jar
- **Java:** 21
- **Dependency:** Spring Web

Gerar, baixar, extrair, abrir no VS Code.

Passo 2 — Pasta do frontend

Criem:

- `src/main/resources/static/index.html`
- `src/main/resources/static/css/estilo.css`
- `src/main/resources/static/js/app.js`

Na `index.html`, não esqueçam:

- Título **StockSales**
- Uma lista para os produtos
- Script do **Axios** antes do `app.js`

Passo 3 — Classe Produto

Pacote: com.entra21.stocksales

```
public class Produto {  
    private Long id;  
    private String nome;  
    private double preco;  
    private int estoque;  
  
    public Produto(Long id, String nome,  
                   double preco, int estoque) {  
        this.id = id;  
        this.nome = nome;  
        this.preco = preco;  
        this.estoque = estoque;  
    }  
  
    public Long getId() { return id; }  
    public String getNome() { return nome; }  
    public double getPreco() { return preco; }  
    public int getEstoque() { return estoque; }
```

Passo 4 — ProdutoController

```
@RestController
public class ProdutoController {

    @GetMapping("/api/produtos")
    public List<Produto> listar() {
        List<Produto> produtos = new ArrayList<>();
        produtos.add(new Produto(1L, "Teclado Mecanico", 250.0, 12));
        produtos.add(new Produto(2L, "Mouse Gamer", 120.0, 30));
        produtos.add(new Produto(3L, "Monitor 24", 900.0, 8));
        produtos.add(new Produto(4L, "Headset", 199.9, 15));
        return produtos;
    }
}
```

Passo 5 — JS consome /api/produtos

No `app.js`:

- 1 Selecionar a lista da página
- 2 `axios.get("/api/produtos")`
- 3 Usar `resposta.data`
- 4 Para cada produto, criar um item na tela com nome, preço e estoque

Testem também a API direto:

`http://localhost:8080/api/produtos`

Como vamos trabalhar daqui pra frente

- Mesmo projeto toda segunda parte da aula
- Cada aula acrescenta uma fatia nova
- Evitem apagar o que já funciona — evoluam
- Salvemos com frequência

Ideia

Vocês não estão fazendo exercício solto.
Estão construindo um produto, aula após aula.

Roteiro sugerido até as 22h

- 1 Criar o projeto no start.spring.io
- 2 Subir e testar que o servidor responde
- 3 Criar `index.html` + CSS mínimo
- 4 Criar `Produto` + `ProdutoController`
- 5 Testar o JSON no navegador
- 6 Ligar o Axios e listar na home

Se travarem no meio: peçam ajuda cedo.

Vocês fecharam a Aula 02 se:

- O StockSales sobe sem erro

- / mostra a página do sistema

- /api/produtos devolve JSON

- A home lista os produtos via Axios

Vocês conseguem explicar:

“a página vem do static; os dados vêm da API”

O que aprendemos hoje

- 1 Estrutura de um projeto Spring Boot
- 2 Diferença entre **página** e **API**
- 3 JSON como formato de dados
- 4 Classe Java com get virando JSON
- 5 Axios no JavaScript para consumir a API
- 6 Como enviar parâmetros com `params`
- 7 Nascimento do projeto **StockSales**

Frases para levar

- A pasta `static/` guarda a vitrine: HTML/CSS/JS.
- A API `/api/...` entrega dados em JSON.
- O JavaScript, com Axios, é a ponte entre a tela e o Spring.
- `params` no Axios conversa com `@RequestParam`.
- Sem banco ainda: lista em memória está ok.
- O StockSales vai crescer até o fim do curso.

Aula 03

- CRUD completo de **produtos**
- CRUD de **clientes**
- Novas telas e formulários
- Ainda em memória, sem banco

Tragam o StockSales funcionando, com listagem de produtos.
Qualquer dúvida de hoje, anotem — começamos revisando.

Obrigado!

Agora é mão na massa no StockSales.

Mauricio Duailibi Neto
Turma Java Entra21